

02. Getting Started with Sorting Problems

Hyunjong Choi, Ph.D.

Assistant Professor of Computer Science

San Diego State University

Outline

❑ Background

- Pseudocode
- Mathematical induction
- Fundamentals of algorithms

❑ Insertion sort ([Textbook 3rd edition 16 – 28](#))

❑ Merge sort (Textbook 3rd edition 29 – 37)

Pseudocode

❑ What is Pseudocode?

- An **artificial and informal language** that helps programmers develop algorithms
- "text-based" detail (algorithmic) design tool

```
INSERTION-SORT( $A$ )
1. for  $j = 2$  to  $A.length$ 
2.      $key = A[j]$ 
3.     // Insert  $A[j]$  into the sorted sequence  $A[1 .. j - 1]$ 
4.      $i = j - 1$ 
5.     while  $i > 0$  and  $A[i] > key$ 
6.          $A[i + 1] = A[i]$ 
7.          $i = i - 1$ 
8.      $A[i + 1] = key$ 
```

< Pseudocode of insertion sort >

Pseudocode conventions

❑ Indention indicates block structure

- Show dependencies of statements
- No begin and end, or “{}”

❑ In loops, i.e., for, while, repeat-until

- loop counter retains its value
- Immediately after the loop, the counter's value is the value that first exceeds the loop bound, e.g., when the for loop terminates, $j == A.length + 1$

INSERTION-SORT(A)

```
1.  for j = 2 to A.length
2.      key = A[j]
3.      // Insert A[j] into the sorted sequence A[1 .. j-1]
4.      i = j - 1
5.      while i > 0 and A[i] > key
6.          A[i + 1] = A[i]
7.          i = i - 1
8.      A[i + 1] = key
```

❑ Access array elements using index in square brackets, e.g, A[i].

- Index starts from 1: A[1], A[2], A[3], ...

❑ Access an attribute of objects using typical syntax for object-oriented languages

- Object name, followed by a dot, followed by the attribute name, e.g., A.length

Pseudocode conventions (cont.)

❑ Arguments are passed by **value**

- The called procedure received its own copy
- If the callee assigns a new value to the argument, the change is not seen by the caller
- Example 1: if x is an argument of a callee procedure, the assignment $x = y$ within the callee procedure is not visible to the caller.
- Example 2: **arrays are passed by pointer**, thus changes to individual array elements are visible to the caller procedure

Pseudocode conventions (cont.)

Notation	Meaning
$x \leftarrow y$	assignment
$=$	comparison of equality
$\neq$	comparison of inequality
<i>true, false</i>	logical true and false
<i>null</i>	nonexistence
$\triangleright$	comment

< General algorithm conventions >

Algorithm A.1 if-then-else structure

```

1: if condition then                                 $\triangleright$  first check
2:   statement1
3:   if condition2 then                                 $\triangleright$  second (nested) if
4:     statement2
5:   end if                                             $\triangleright$  end of second if
6: else if condition3 then                             $\triangleright$  else if of first if
7:   statement3
8: else
9:   statement4
10: end if                                             $\triangleright$  end of first if

```

Notation	Meaning
$\neg$	logical negation
$\wedge$	logical and
$\vee$	logical or
$\oplus$	logical exclusive-or
x/y	x divided by y
$x \cdot y$ or xy	multiplication

< Arithmetic and logical operators >

- More conventions :

<https://link.springer.com/content/pdf/bbm%3A978-1-4471-5173-9%2F1.pdf>

Mathematical induction

□ Overview

- A proof technique used to establish that a given statement holds for all natural numbers (or a specific subset of them).
 - ✓ Step 1 (**Base Case**): show that the statement holds for the initial value (usually $n = 1$).
 - ✓ Step 2 (**Inductive Step**): assume the statement is true for some arbitrary integer $n = k$ (this is called the *inductive hypothesis*), and then prove it for $n = k + 1$.



If both steps are successfully proven, the statement is true for all integers n greater than or equal to the base case.

Mathematical induction (example)

□ Sum of the first n natural numbers

- Prove that the sum of the first n natural numbers is given by the formula:

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

- Step 1: base case ($n = 1$)

✓ For $n = 1$, the left-hand side is $1 = \frac{1(1+1)}{2} = 1$. Thus, the base case holds.

- Step 2: inductive step

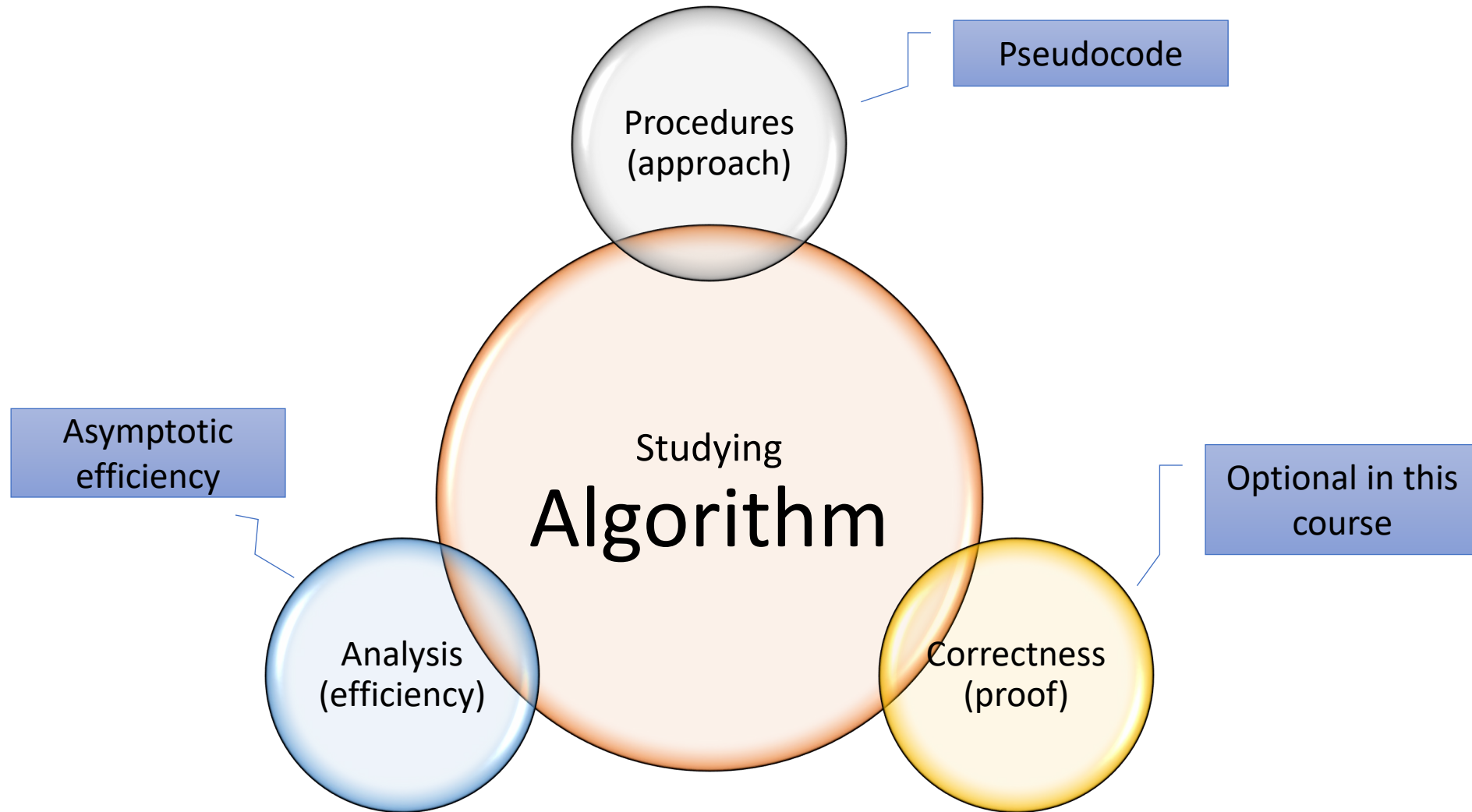
✓ Inductive hypothesis:

○ Assume the formula is true for $n = k$, i.e., $1 + 2 + 3 + \cdots + k = \frac{k(k+1)}{2}$

✓ We need to prove that the formula holds for $n = k + 1$, meaning:

$$1 + 2 + 3 + \cdots + k + (k + 1) = \frac{(k+1)(k+2)}{2}$$
$$\frac{k(k+1)}{2} + (k+1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2}$$

Fundamentals of algorithm



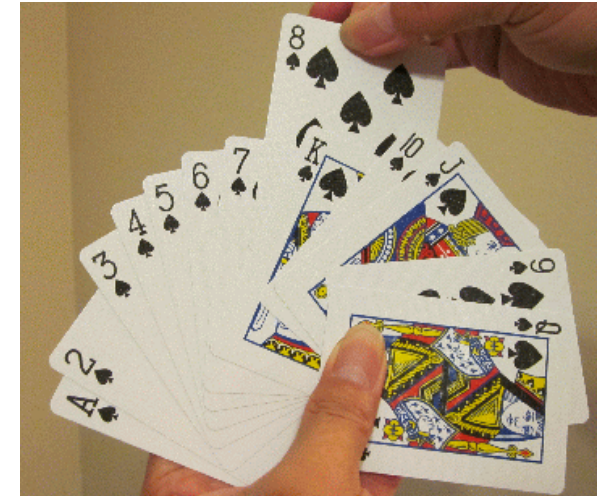
Insertion sort overview

❑ Problem

- Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$
- Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

❑ Insertion sort: similar to the way of sorting poker cards

- A pile of cards facing down on the table
- Start with an **empty** hand
- Take **one card** at a time from the table and **insert** it into the **correct** position in the hand
- To find the correct position, we compare it with each of the cards already in the hand



Step-wise analysis

INSERTION-SORT(A)

```
1. for  $j = 2$  to  $A.length$ 
2.    $key = A[j]$ 
3.   // Insert  $A[j]$  into the sorted sequence  $A[1 .. j - 1]$ 
4.    $i = j - 1$ 
5.   while  $i > 0$  and  $A[i] > key$ 
6.      $A[i + 1] = A[i]$ 
7.      $i = i - 1$ 
8.    $A[i + 1] = key$ 
```

With input:

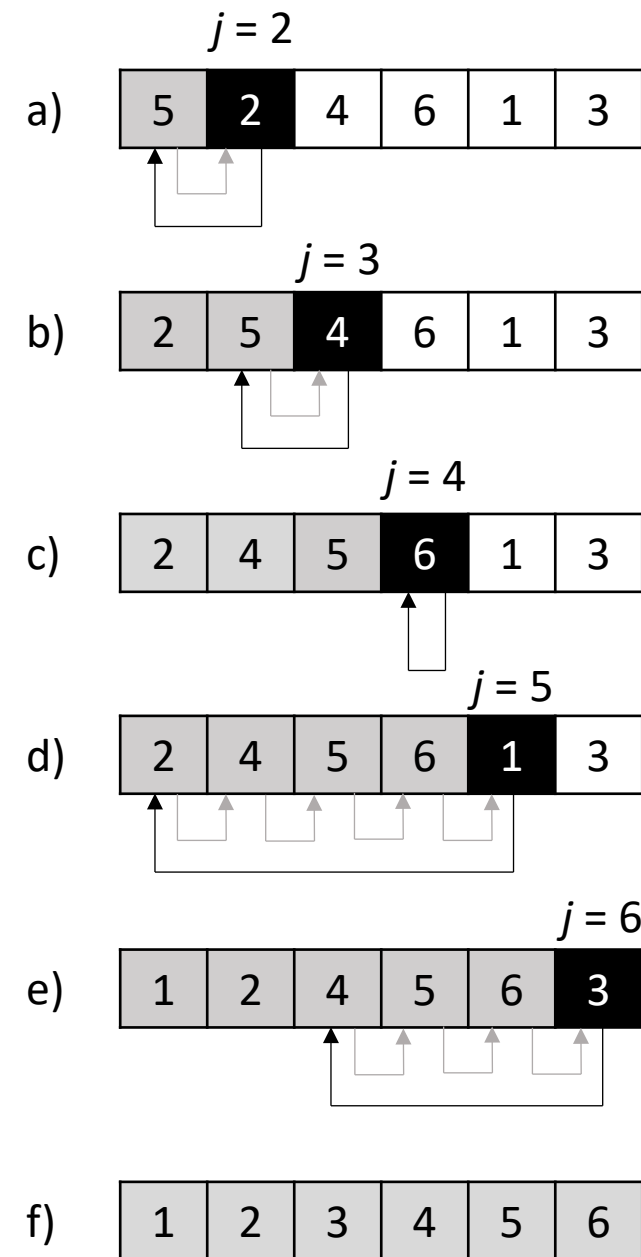
$A = \langle 5, 2, 4, 6, 1, 3 \rangle$

$A.length = 6$

Sorts **in place**: rearranges
the numbers within A



No extra memory costed



Analogy to sorting poker cards

INSERTION-SORT(A)

```
1. for j = 2 to A.length
2.   key = A[j]
3.   // Insert A[j] into the sorted sequence A[1 .. j-1]
4.   i = j - 1
5.   while i > 0 and A[i] > key
6.     A[i + 1] = A[i]
7.     i = i - 1
8.   A[i + 1] = key
```

j indicates the “current card” being inserted

$A[1:j - 1]$ are the currently sorted cards

$A[j + 1:n]$ are the remaining pile of cards

Use *loop invariant* to check the correctness

- ❑ **Loop invariants** help us understand **why an algorithm is correct**
- ❑ Need to show three things about a loop invariant
 - **Initialization:** It is true prior to the first iteration of the loop
 - **Maintenance:** If it is true before an iteration of the loop, it remains true before the next iteration
 - **Termination:** When the loop terminates, the invariant gives a useful property that shows the algorithm is correct

“ $A[1:j - 1]$ are the elements originally in positions 1 through $j-1$, in sorted order.”

Loop invariant for insertion sort

Use *loop invariant* to check the correctness

□ Initialization: Before the first loop iteration

- $j = 2$, the subarray $A[1:j-1]$ consists of the single element $A[1]$, and it is sorted.
- Therefore, loop invariant holds.

□ Maintenance:

- Before an iteration, $A[1:j-1]$ is already sorted
- The body of for loop moves $A[j-1]$, $A[j-2]$, $A[j-3]$... by one position to the right, until it finds the proper position for $A[j]$ (Line 4 ~ 7)
- At which point it inserts the value of $A[j]$ (Line 8)
- Now the subarray $A[1:j]$ consists of the elements originally in $A[1:j]$, but in sorted order.
- Incrementing j for the next iteration then **preserves** the loop invariant

```
INSERTION-SORT(A)
1.  for  $j = 2$  to  $A.length$ 
2.       $key = A[j]$ 
3.      // Insert  $A[j]$  into the sorted sequence  $A[1 .. j-1]$ 
4.       $i = j - 1$ 
5.      while  $i > 0$  and  $A[i] > key$ 
6.           $A[i + 1] = A[i]$ 
7.           $i = i - 1$ 
8.       $A[i + 1] = key$ 
```

A more formal examination requires us to show a loop invariant for the **while** loop as well

Use *loop invariant* to check the correctness

□ Termination:

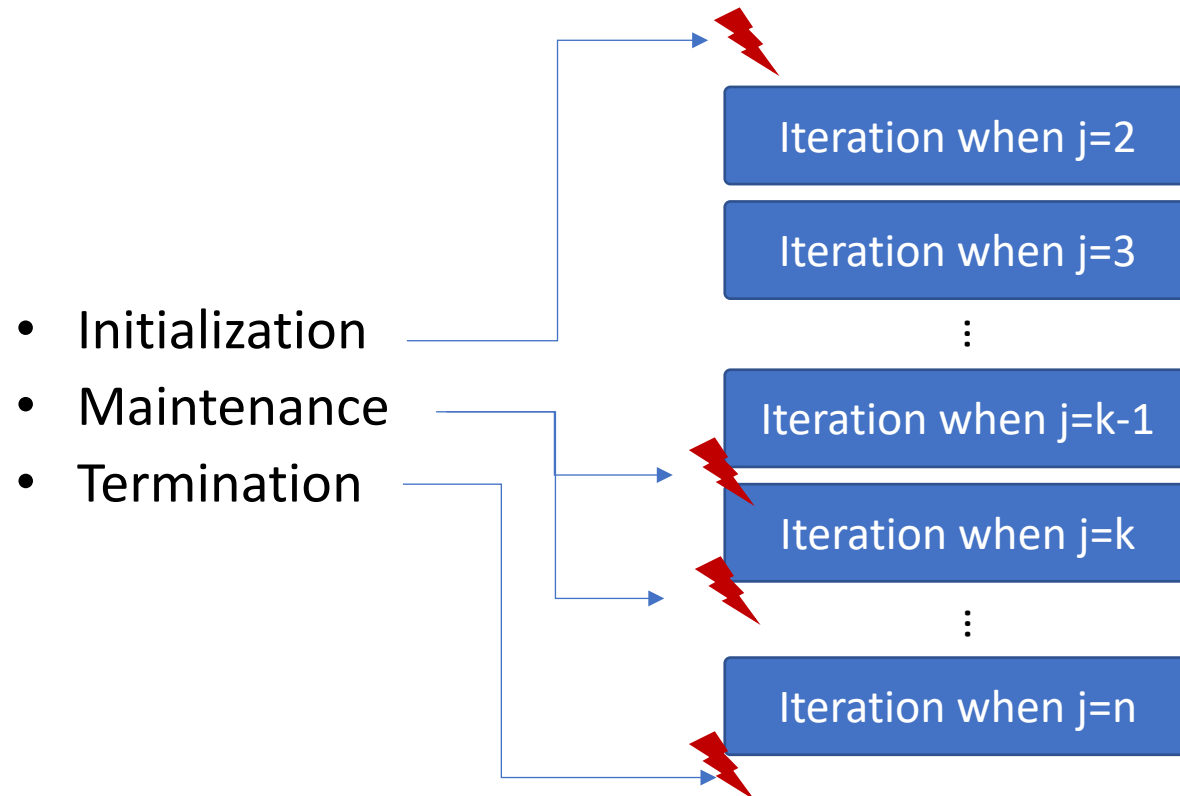
- The condition causing the **for** loop to terminate is $j > A.length = n$
- We must have $j = n + 1$ when the loop terminates
- Substituting $n+1$ for j into the loop invariant
- We have that subarray $A[1:n]$ consists of the elements originally in $A[1:n]$, but in sorted order
- This subarray is the entire array A itself, therefore the **algorithm is correct**.

```
INSERTION-SORT(A)
1.  for  $j = 2$  to  $A.length$ 
2.       $key = A[j]$ 
3.      // Insert  $A[j]$  into the sorted sequence  $A[1 .. j-1]$ 
4.       $i = j - 1$ 
5.      while  $i > 0$  and  $A[i] > key$ 
6.           $A[i + 1] = A[i]$ 
7.           $i = i - 1$ 
8.       $A[i + 1] = key$ 
```

Summarize loop invariant for insertion sort

□ Loop invariant

- Insertion sort: “ $A[1:j-1]$ are the elements originally in positions 1 through $j-1$, in sorted order.”



```
INSERTION-SORT(A)
1.  for  $j = 2$  to  $A.length$ 
2.       $key = A[j]$ 
3.      // Insert  $A[j]$  into the sorted sequence  $A[1 .. j-1]$ 
4.       $i = j - 1$ 
5.      while  $i > 0$  and  $A[i] > key$ 
6.           $A[i + 1] = A[i]$ 
7.           $i = i - 1$ 
8.       $A[i + 1] = key$ 
```

Assumptions for analyzing efficiency

❑ What we assume to measure efficiency of algorithm?

- Disable hyper-threading (if Intel CPUs)
- Assume a simple/generic one-processor model of computer
- Set max. frequency of CPU / Disable DVFS
- Instructions are executed sequentially; no concurrent/parallel ops

❑ A common set of “primitive” instructions:

- Arithmetic: add (+), subtract (-), multiply (*), divide (/), remainder (%), floor, ceiling.
- Data movement: load, store, copy
- Control: conditional branch (if ... else), subroutine call and return

 **These instructions take a *constant* amount of time.**

Defining running time

- ❑ The time taken by INSERTION-SORT depends on the input:
 - Longer time to sort longer input arrays
 - Shorter time for input arrays that are already nearly sorted
- ❑ In general, the time taken grows with the size of input
- ❑ We can describe the running time of an algorithm as *a function of the input size*

$$T(n)$$

- ❑ **Input size:** number of items in input, e.g., the number of items being sorted, etc.
- ❑ **Running time:** the number of primitive instructions or “steps” executed
 - A constant amount of time is required to execute each line of pseudocode.

Analysis of insertion sort

- ❑ Assume each execution of the ***i*-th line takes c_i time** (a constant cost, in *seconds*).
- ❑ Each line is executed a certain number of ***times*** (integer numbers)

INSERTION-SORT(A)		Cost	Number of times	
1.	for $j = 2$ to $A.length$	c_1	n	After the last iteration, $j = n+1$
2.	$key = A[j]$	c_2	$n - 1$	
3.	// Insert $A[j]$ into the sorted sequence $A[1 .. j-1]$	0	$n - 1$	
4.	$i = j - 1$	c_4	$n - 1$	How many times are the while statement executed?
5.	while $i > 0$ and $A[i] > key$	c_5	?	
6.	$A[i + 1] = A[i]$	c_6	?	
7.	$i = i - 1$	c_7	?	
8.	$A[i + 1] = key$	c_8	$n - 1$	For each j ($j = 2, \dots, n$), the while loop test is executed t_j times, which is <i>unknown</i>

Therefore, the total number of times is $\sum_{j=2}^n t_j$

Analysis of insertion sort

INSERTION-SORT(A)	Cost	Number of times
1. for $j = 2$ to $A.length$	c_1	n
2. $key = A[j]$	c_2	$n - 1$
3. // Insert $A[j]$ into the sorted sequence $A[1 .. j-1]$	0	$n - 1$
4. $i = j - 1$	c_4	$n - 1$
5. while $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^n t_j$
6. $A[i + 1] = A[i]$	c_6	$\sum_{j=2}^n (t_j - 1)$
7. $i = i - 1$	c_7	$\sum_{j=2}^n (t_j - 1)$
8. $A[i + 1] = key$	c_8	$n - 1$

Same reason as the for loop

❑ Total running time $T(n) \Rightarrow$ the sum of running times for each statement executed

❑ A statement that costs c_i to execute and executes n times will contribute $c_i n$ to the total

$$T(n) = c_1 n + c_2 (n - 1) + c_4 (n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8 (n - 1)$$

Running time of insertion sort: best-case

□ The best case: input array is already sorted.

- For any $i < j$, we always have $A[i] \leq A[j]$
- $A[i] \leq \text{key}$ always breaks the while loop (Line 5)
- Thus, $t_j = 1$ for $j = 2, 3, \dots, n$

```
INSERTION-SORT(A)
1.  for j = 2 to A.length
2.      key = A[j]
3.      // Insert A[j] into the sorted sequence A[1 .. j-1]
4.      i = j - 1
5.      while i > 0 and A[i] > key
6.          A[i + 1] = A[i]
7.          i = i - 1
8.      A[i + 1] = key
```

$$T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1)$$

➡
$$\begin{aligned} T(n) &= c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

$T(n) = an + b$, where a and b are constants

➡ $T(n)$ is a **linear function** of n (for the best case)

Running time of insertion sort: worst-case

- ❑ The worst case is that the input array is in reverse sorted order
 - The key = $A[j]$ needs to be compared with each element in $A[1 \dots j - 1]$
 - Thus, $t_j = j$ for $j = 2, 3, \dots, n$

❑ Note that: $\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$, $\sum_{j=2}^n (j - 1) = \frac{n(n-1)}{2}$

$$\begin{aligned} T(n) &= c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \left(\frac{n(n + 1)}{2} - 1 \right) + c_6 \left(\frac{n(n - 1)}{2} \right) + c_7 \left(\frac{n(n - 1)}{2} \right) + c_8(n - 1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

Simplified as $an^2 + bn + c$

➡ $T(n)$ is a **quadratic function** of n (in the worst case)

Worst-case and average-case analysis

❑ Worst-case running time:

- It gives an upper bound for any input (will never take any longer)
- The worst case occurs fairly often

❑ For insertion sort, the **average case** as bad as the worst case

- Suppose n random numbers as input
- Question: what is t_j (on average)?
 - ✓ A fair guess: half the elements in $A[1:j-1]$ are less than $A[j]$, and the other half are greater
- Thus, t_j is about $j/2$
- $T(n) = \sum_{j=2}^n t_j = \sum_{j=2}^n j/2 = \frac{1}{2} \sum_{j=2}^n j$
- $T(n)$ is also a quadratic function of n .

Example

Problem

- Calculate the total number of iterations for the given **for** loop structure, i.e., how many times Line 4 executed?

Algorithm Loop (n):

L1. $s = 0$

L2. For $i = 1$ to $2n$ do

L3. For $j = i$ to $2n$ do

L4. $s = s + i$

Q & A

